

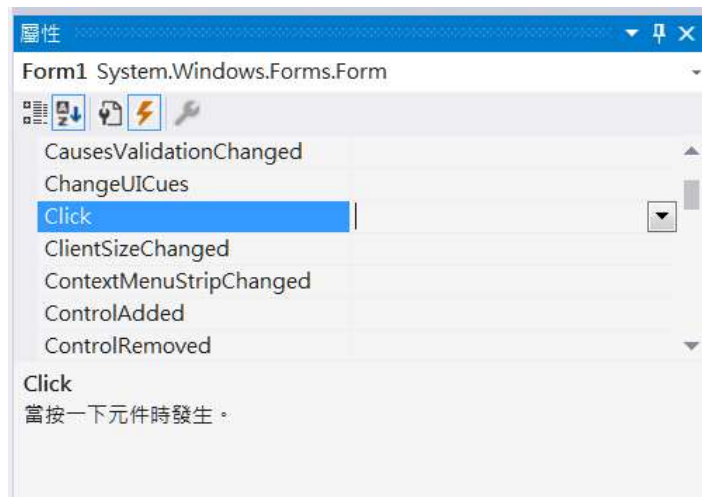
# .NET平台的事件模式

- **8-1:製作註冊事件處理方法程式碼**
  - 註冊事件.....8-1
  - 製作註冊事件的程式碼.....8-2
- **8-2:事件委派 (delegate) 機制**
  - 簡說事件委派(delegate).....8-4
- **8-3:事件驅動程式設計– Event-driven programming**
  - Event-driven programming.....8-6

*Module 08*

# 註冊事件

- 在[設計]視窗選取物件後點擊右鍵，按下[屬性]叫出屬性視窗，在屬性視窗的頂端按下 ⚡，進入事件視窗。
- 事件視窗羅列該物件的所有事件，選取事件時，下方會出現該事件的說明文字。
- 選取事件後快點兩下，Visaul Studio會自動建立註冊事件。



```
private void Form1_Click(object sender, System.EventArgs e)
{
    //撰寫Click Form1後發生的事
}
```

# 製作註冊事件的程式碼-1

- 所有的事件都必須註冊才能使用。
- 當在事件視窗快點兩下時，**Visual Studio**會自動在**.Designer.cs**檔中，自動加上註冊程式碼。

```
private void InitializeComponent()
{
    this.SuspendLayout();
    //
    // Form1
    //
    this.AutoScaleDimensions = new System.Drawing.SizeF(8F, 15F);
    this.AutoScaleMode = System.Windows.Forms.AutoScaleMode.Font;
    this.ClientSize = new System.Drawing.Size(800, 450);
    this.Name = "Form1";
    this.Text = "Form1";
    this.Click += new System.EventHandler(this.Form1_Click);
    this.ResumeLayout(false);
}
```

## 製作註冊事件的程式碼-2

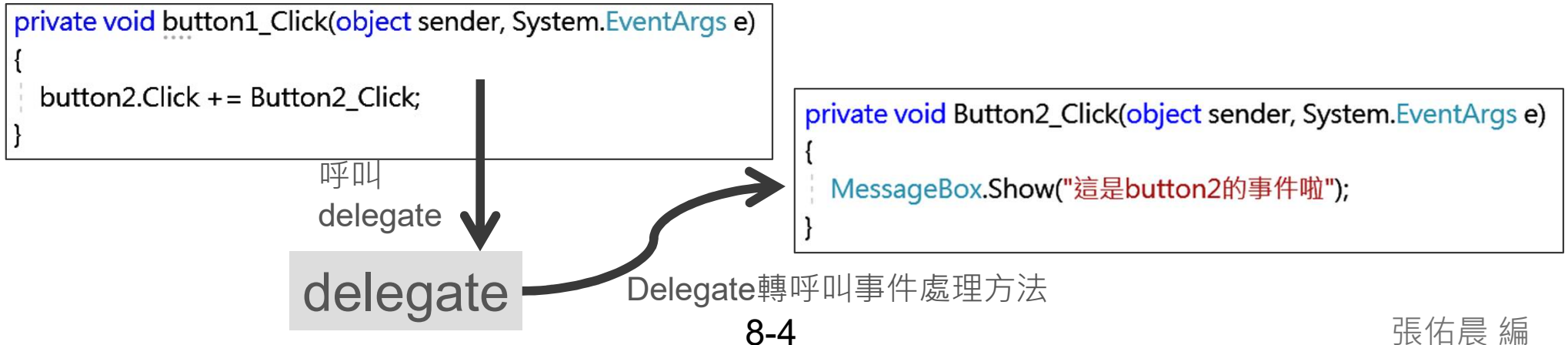
- 自行撰寫註冊事件。

```
private void button1_Click(object sender, System.EventArgs e)
{
    button2.Click += Button2_Click;
}
```

```
private void Button2_Click(object sender, System.EventArgs e)
{
    MessageBox.Show("這是button2的事件啦");
}
```

# 簡說事件委派(delegate)-1

- 委派(delegate)：
  - 用來定義方法的格式(傳回型態、參數)。
  - 可視為「方法的參考型別」，能儲存並呼叫符合定義的方法。
- 透過委派來定義可繫結的方法，用於觸發特定動作或通知其他物件。
- 使用 += 來訂閱事件，將對應的方法與事件繫結。（事件觸發時，會自動呼叫所註冊的方法）。



## 簡說事件委派(delegate)-2

- 委派可以指向多個方法。

```
delegate double Payment(double money);

double Discount50off(double money)
{
    return money * 0.5;
}

double Discount20off(double money)
{
    return money * 0.8;
}
```

```
private void btnClac_Click(object sender, EventArgs e)
{
    Payment pay;
    if (cmbDiscount.Text == "50off")
    {
        pay = Discount50off;
    }
    else
    {
        pay = Discount20off;
    }
    double FinalCost = pay(Convert.ToDouble(txtMoney.Text));
    labFinalCost.Text = FinalCost.ToString();
}
```

# Event-driven programming

- 程式依據事件的發生來決定執行的動作，而非按固定流程運作。
- 系統會持續監聽外部事件，並在事件發生時觸發對應的函式或方法進行處理。
- 事件來源可包括：使用者操作（滑鼠、鍵盤）、檔案或網路輸入、計時器或系統通知。
  - 「事件 → 觸發 → 對應方法執行」
  - 微軟的Windows是典型的事件驅動架構作業系統。